

IFS Test Automation

IFS Academy



Agenda:

01

Importance of Test Automation

02

Test-A-Rest Tool

03

Test Category Types

04

Test Script Structure

05

Test-A-Rest Basic Examples

06

JSON Manipulation

Course Objectives

After following this training, the participants will have a clear understanding of,

- The need of writing Test Automation scripts
- How to write Test Automation scripts and execute using Test-A-Rest tool.
- Different types of JSON Manipulations

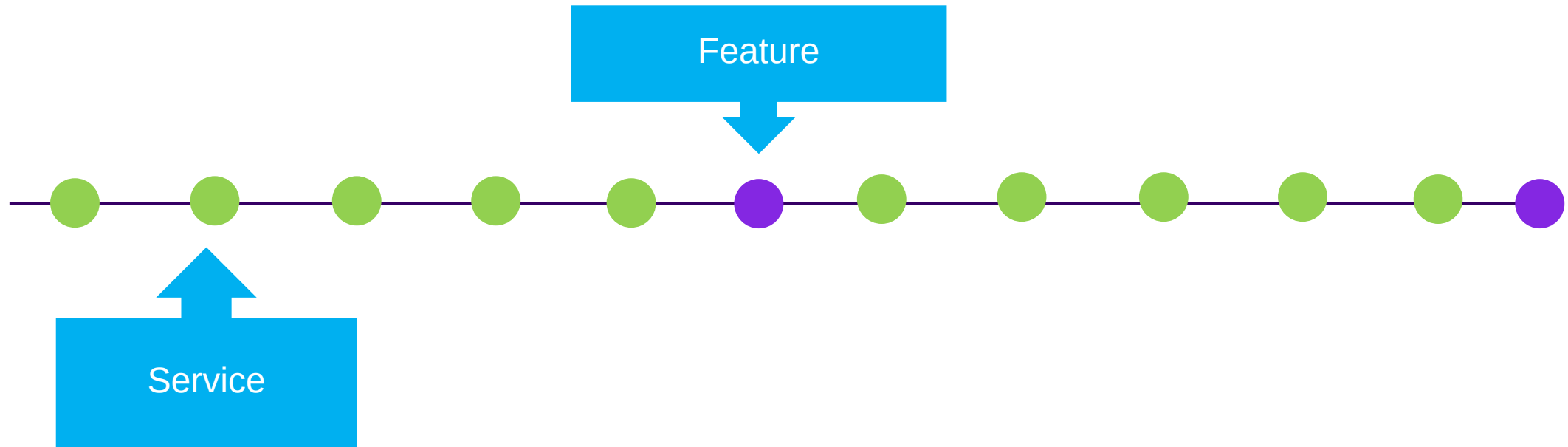


Importance of Test Automation



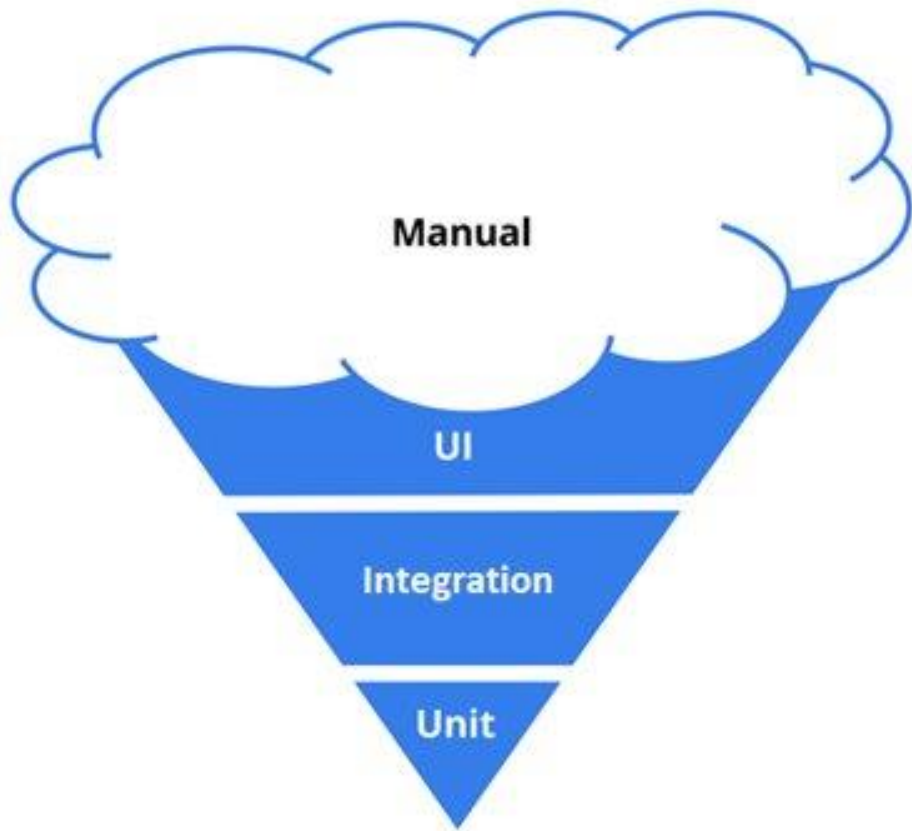
Importance of Test Automation

Release cadence ~ 6 months



Importance of Test Automation

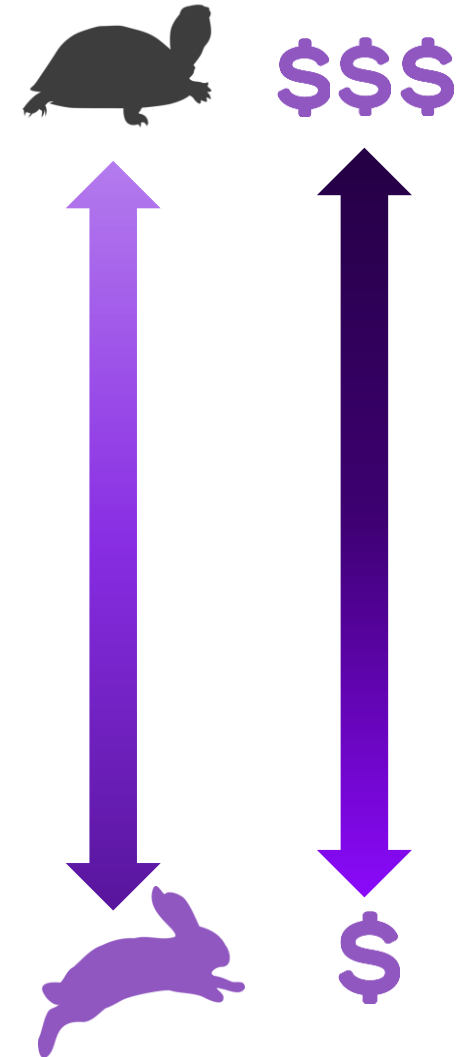
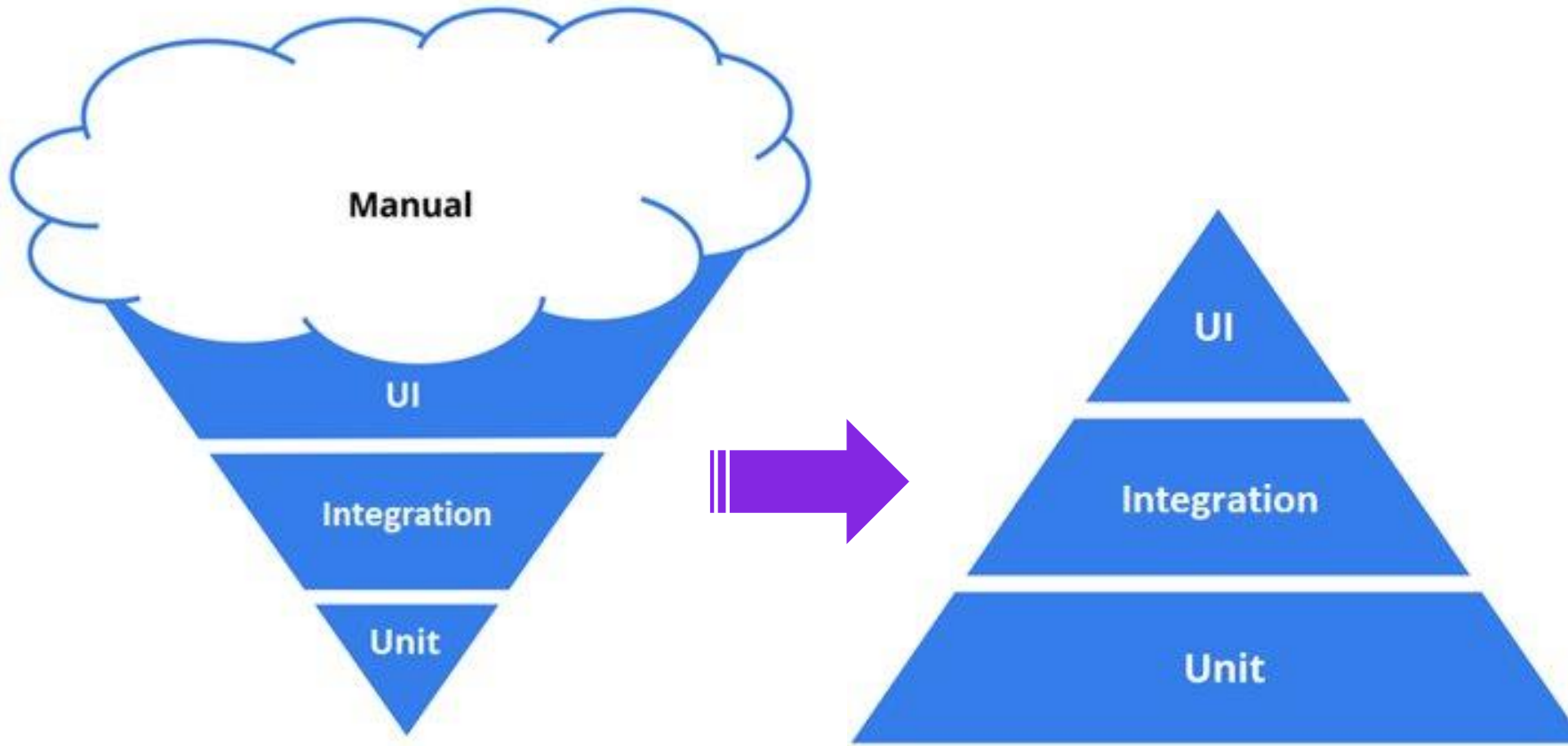
Test Pyramid before the IFS Aurena Framework



- Used to have lot business logic written in the IFS Enterprise Explorer client
- Lot of manual testing carried out in the UI level
- Limitations on testing a business flow without the UI interactions

Importance of Test Automation

Transformation of the Test Pyramid using Test Automation

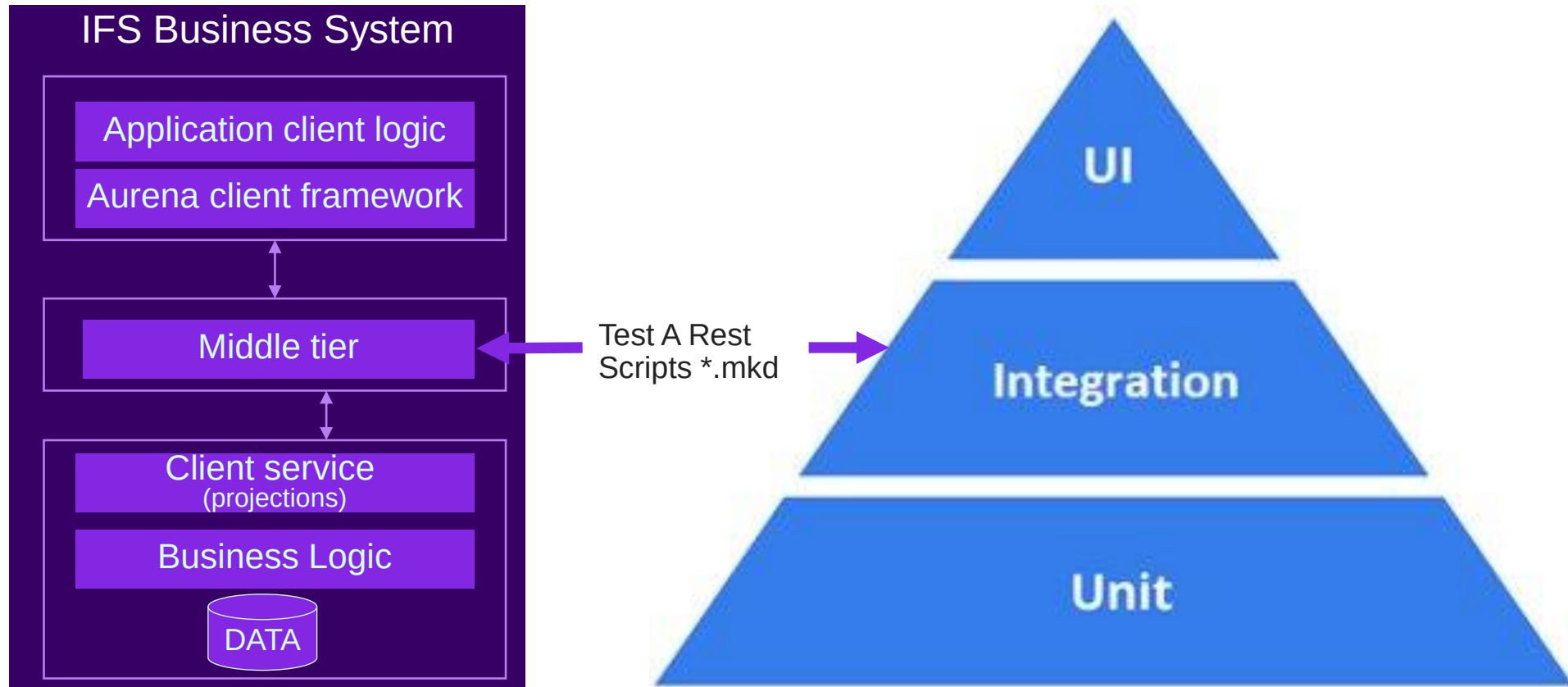


Test-A-Rest Tool

Test-A-Rest Tool

Introduction to Test-A-Rest Tool

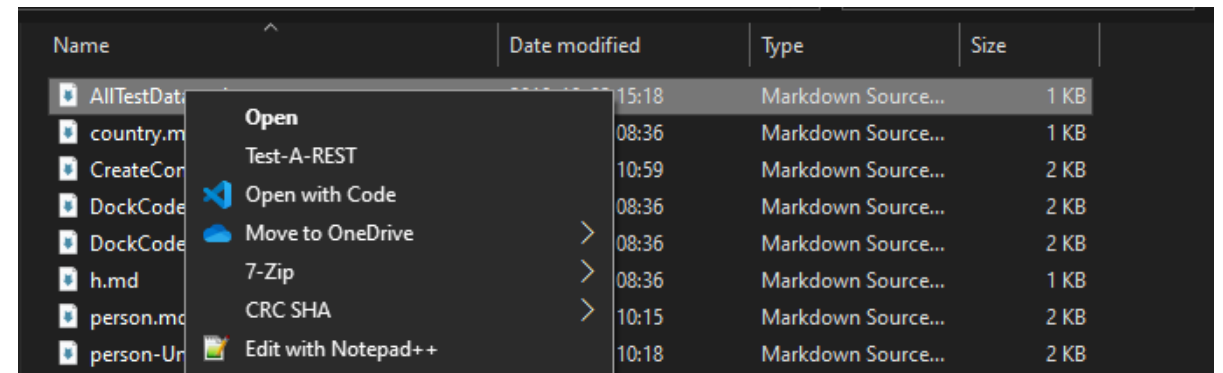
- Developed by IFS internally
- Less complex programming style
- No Licensing cost
- Can be version controlled
- Based on Microsoft .Net CORE
- Command Line Tool
- Markdown, Json and C# used as technologies



Test-A-Rest Tool

Getting Started

1. Download the Test-A-Rest tool from the IFS Cloud Build Place and unpack it to a suitable location on your machine
2. Run the `_install.cmd` file to register a Test-A-Rest context menu.
 - a. Edit the `_test_a_rest.cmd` file to specify server and credentials if you prefer any other server than the default
3. Try the samples using the Test-A-Rest Context menu
4. Recommended editor is Visual Studio Code
5. Create a Markdown text file(.mkd)



Test-A-Rest Tool

Markdown file

- Test scripts are written using markdown(.mkd) text files
- It's a plain text format for writing structured documents based on formatting conventions. Refer [here](#) for more information.

Type this	...To Get This
<code>*Italic*</code>	<i>Italic</i>
<code>**Bold**</code>	Bold
<code># Heading 1</code>	Heading 1
<code>## Heading 2</code>	Heading 2
<code>[Link](https://www.ifs.com)</code>	Link
<code>* List</code> <code>* List</code> <code>* List</code>	• List • List • List

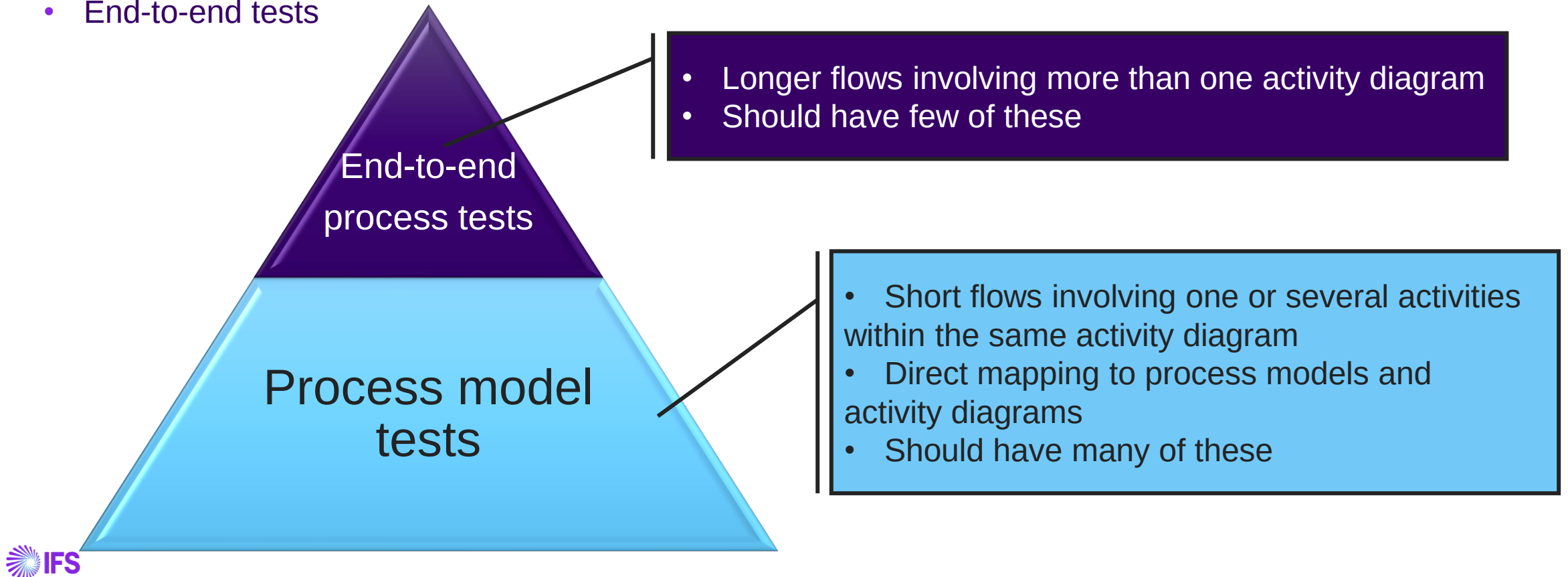
Test Category Types



Test Category Types

Test categories

- All tests to be executed out with IFS Test-A-Rest are integration tests
- Within the integration test concept, we currently have two different categories of tests:
 - Process Model Tests
 - End-to-end tests



Test Category Types

Process Model Test Types

Test Collection

A way of categorizing Test Suites

Corresponds to an activity diagram

Test Suite

A way of categorizing a collection of *test cases*

activity in a process model

Since our process model activities vary in size, an activity test suite can be anything from one to many test cases

Test Case

Consists of a number of *test steps*

Corresponds to a series of instructions found inside a process model activity

Also contains the test specification. So the test specification and the implemented test case reside within the same markdown file

Test Utils

Helper classes that can be reused by other test cases.

Utils should not contain any asserts and should be standalone.

Test Data

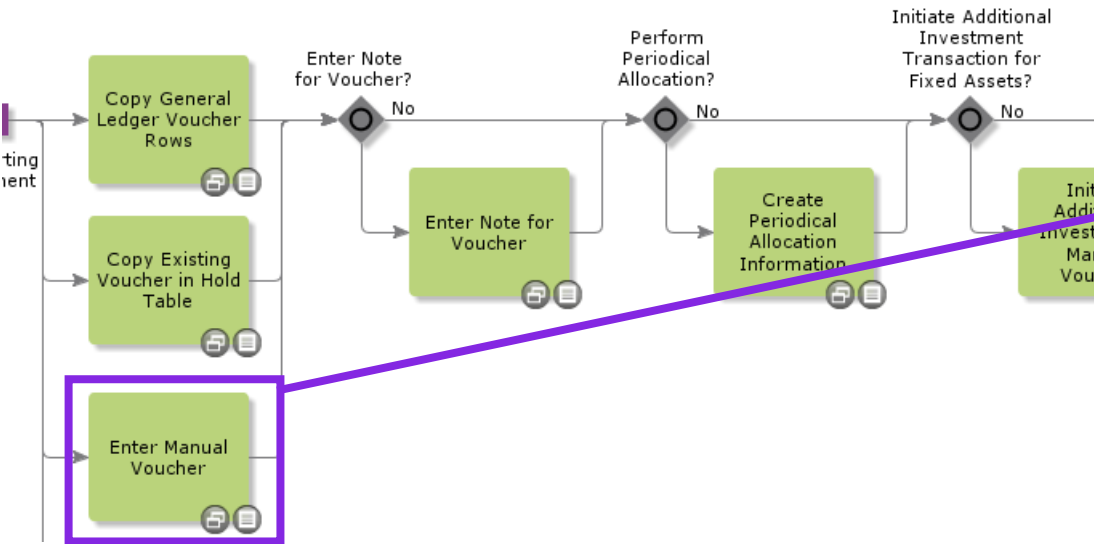
Simple test data scripts in markdown file format

Only markdown files, i.e. no other scripts or manual actions needed

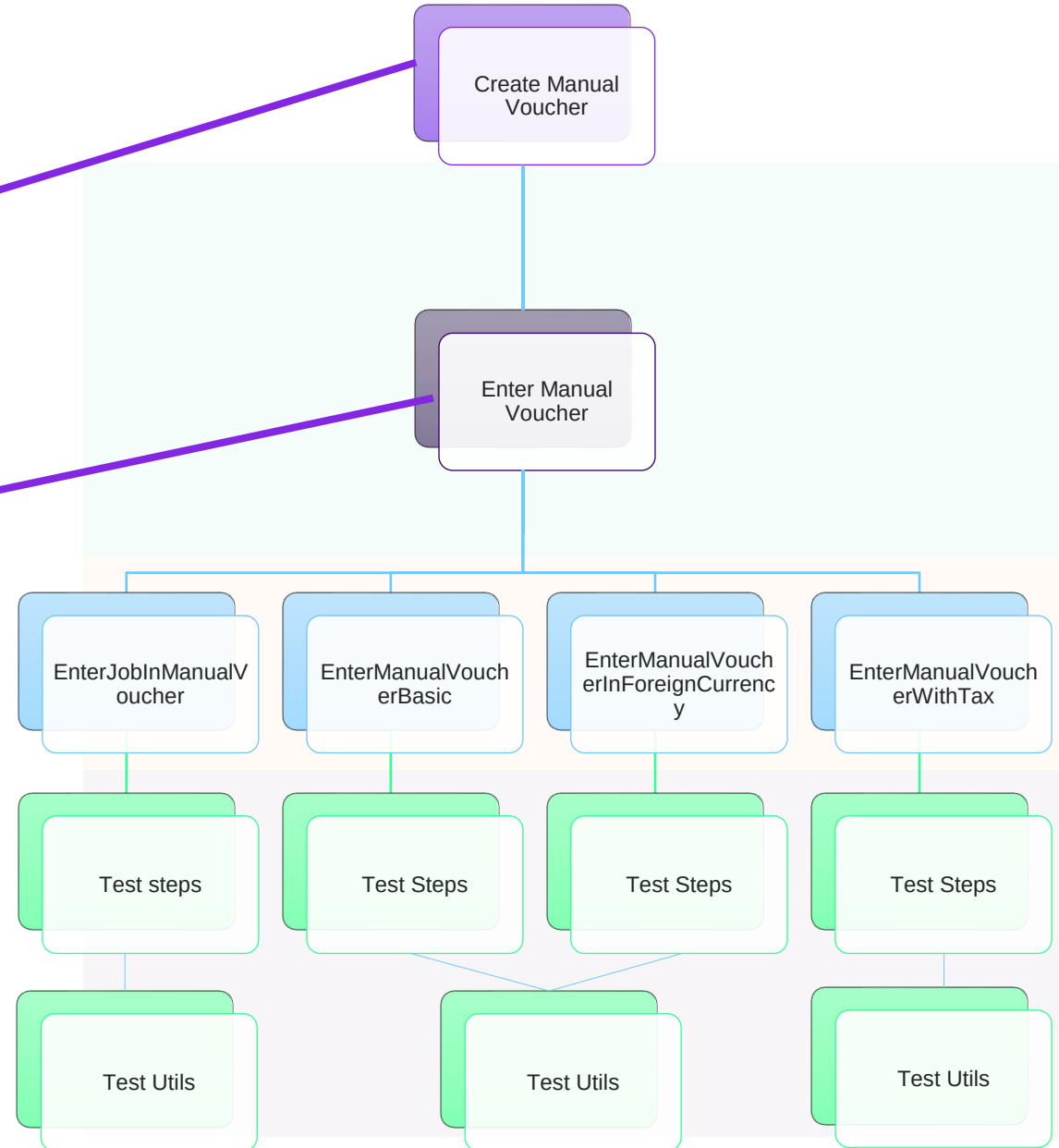
Test Category Types

Process Model Tests - Example

2.2.3.2. Create Manual Voucher



- Direct mapping to process models, in this case activity diagram “2.2.3.2 Create Manual Voucher”



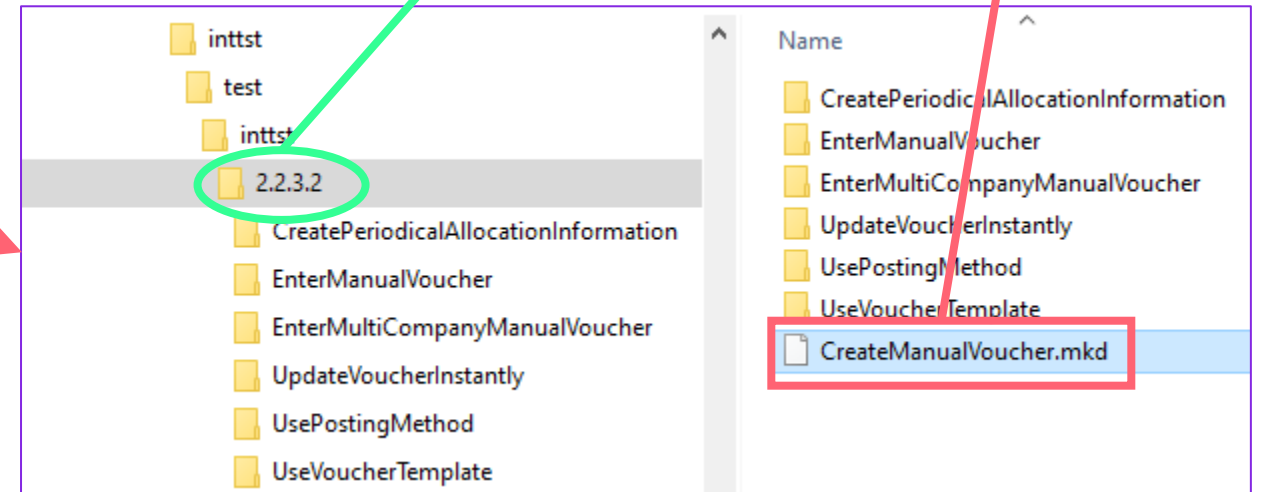
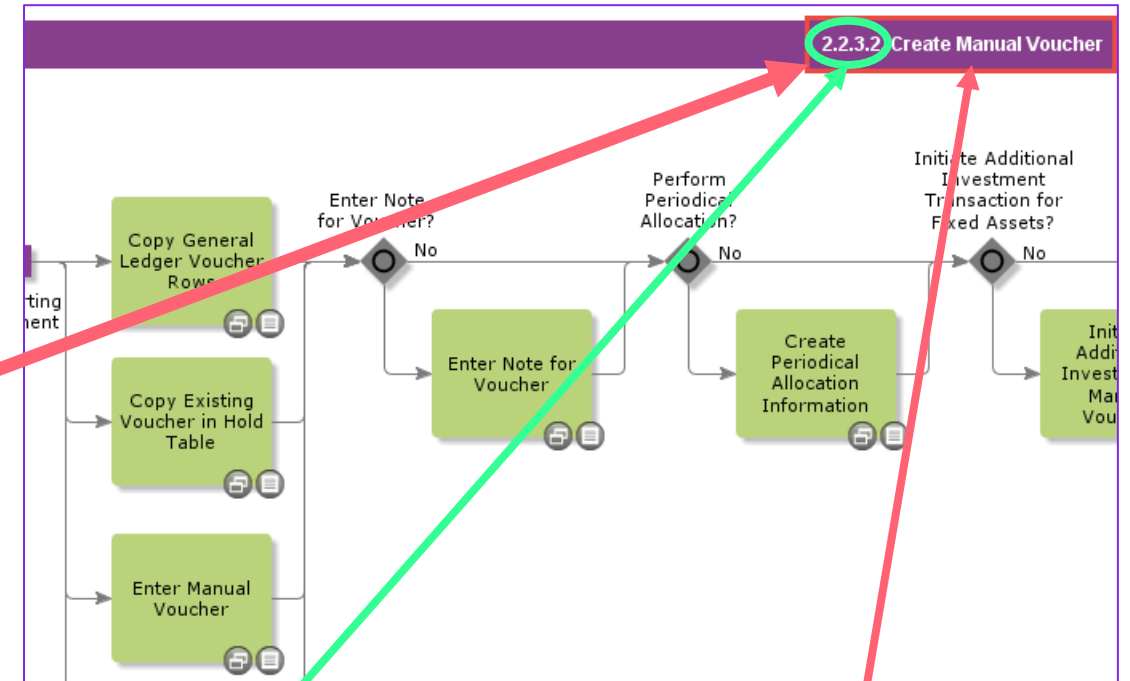
Test Category Types

Process Model maps to file structure

Test Collection

A way of categorizing Test Suites

Corresponds to an activity diagram



Test Category Types

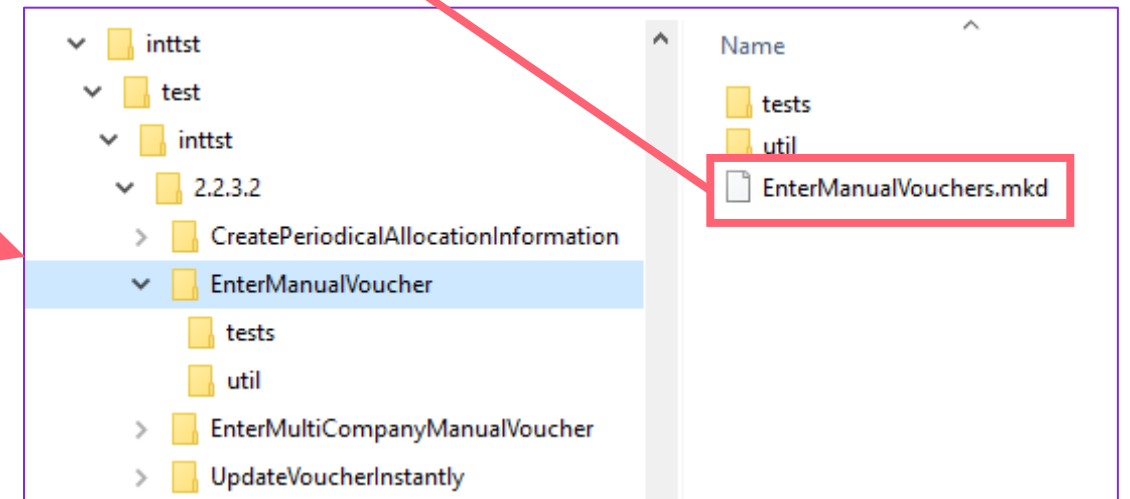
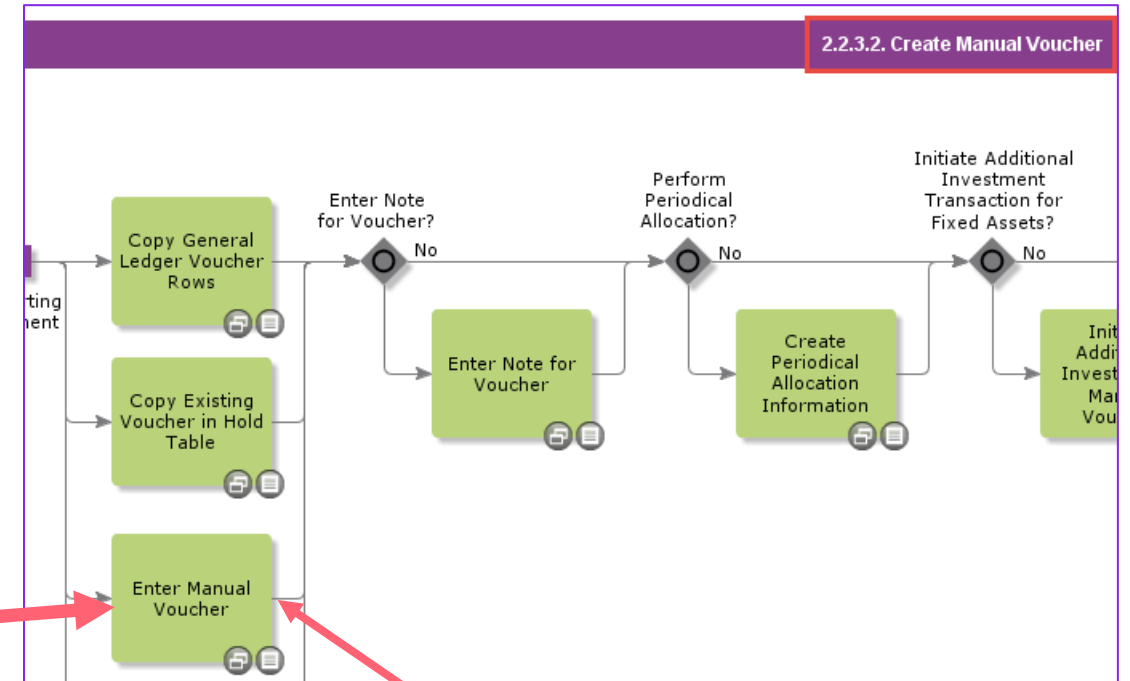
Process Model maps to file structure

Test Suite

A way of categorizing a collection of test cases

activity in a process model

Since our process model activities vary in size, an activity test suite can be anything from one to many test cases



Test Category Types

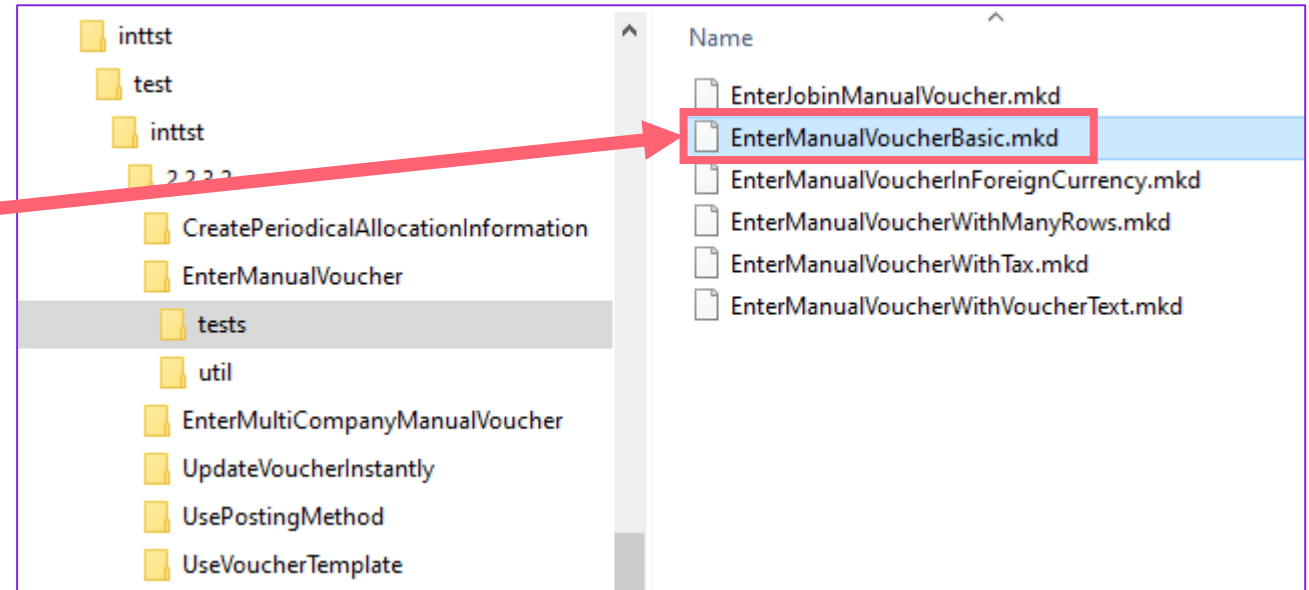
Process Model maps to file structure

Test Case

Consists of a number of *test steps*

Corresponds to a series of instructions found inside a process model activity

Also contains the test specification. So the test specification and the implemented test case reside within the same markdown file



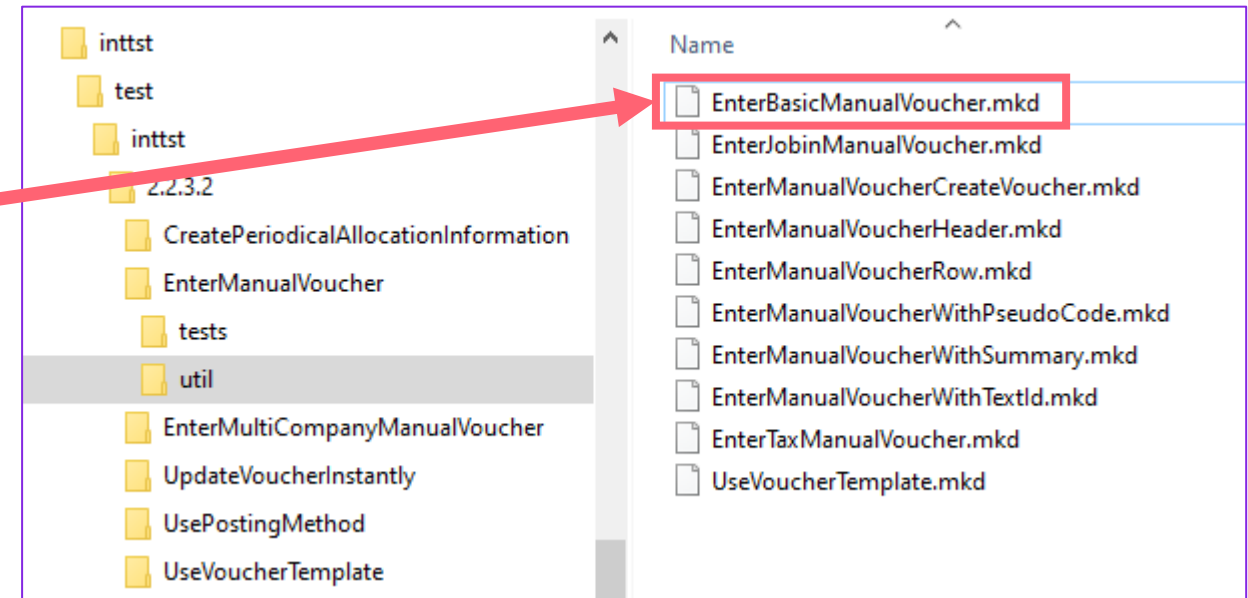
Test Category Types

Process Model maps to file structure

Test Utils

Helper classes that can be reused by other test cases.

Utils should not contain any asserts and should be standalone.



Test Script Structure



Test Script Structure

Main sections of the Test-A-Rest script

You can mix meta data, text and code in same document

Metadata	1 --- 2 type: Test Case 3 owner: [Product Area] 4 mode: [Standalone/Dependent] 5 --- 6
Test specification	7 # X.X.X.X [Activity Diagram name] 8 9 ## Description 10 What is this Test Case about. 11
Code	12 ```cs 13 // Add your code here 14 ```

- The mode of the test cases should be standalone unless there is an exceptional situation to declare them as dependent.
- Characteristics of Standalone test cases:
 - No prerequisites except test data
 - Test cases can be executed in any order
 - Rerunnable – Should clean up data after the test case is run or randomize data

Test Script Structure

Main sections of the Test-A-Rest script

Meta Data

Meta data is used to detect what type of test file it is, which product area that owns the test and if it can be executed without being dependent on any other test

Metadata

```
1  ---
2  type: Test-Case
3  owner: [Product-Area]
4  mode: [Standalone/Dependent]
5  ---
6
7  # X.X.X.X [Activity-Diagram-name]
8
9  ## Description
10 What-is-this-Test-Case-about.
11
12 ````cs
13 // Add your code here
14 ```
```

Test Script Structure

Main sections of the Test-A-Rest script

Test Specification

The text part is used for describing what the test is about for anyone.

The text always starts with a heading. You can have several text sections in the document. See templates for what text is mandatory to have

Text

```
1  ---
2  type: Test-Case
3  owner: [Product-Area]
4  mode: [Standalone/Dependent]
5  ---
6
7  # X.X.X.X [Activity-Diagram-name]
8
9  ## Description
10 What-is-this-Test-Case-about.
11
12 ```cs
13 // Add your code here
14 ```
```

Test Script Structure

Main sections of the Test-A-Rest script

Code

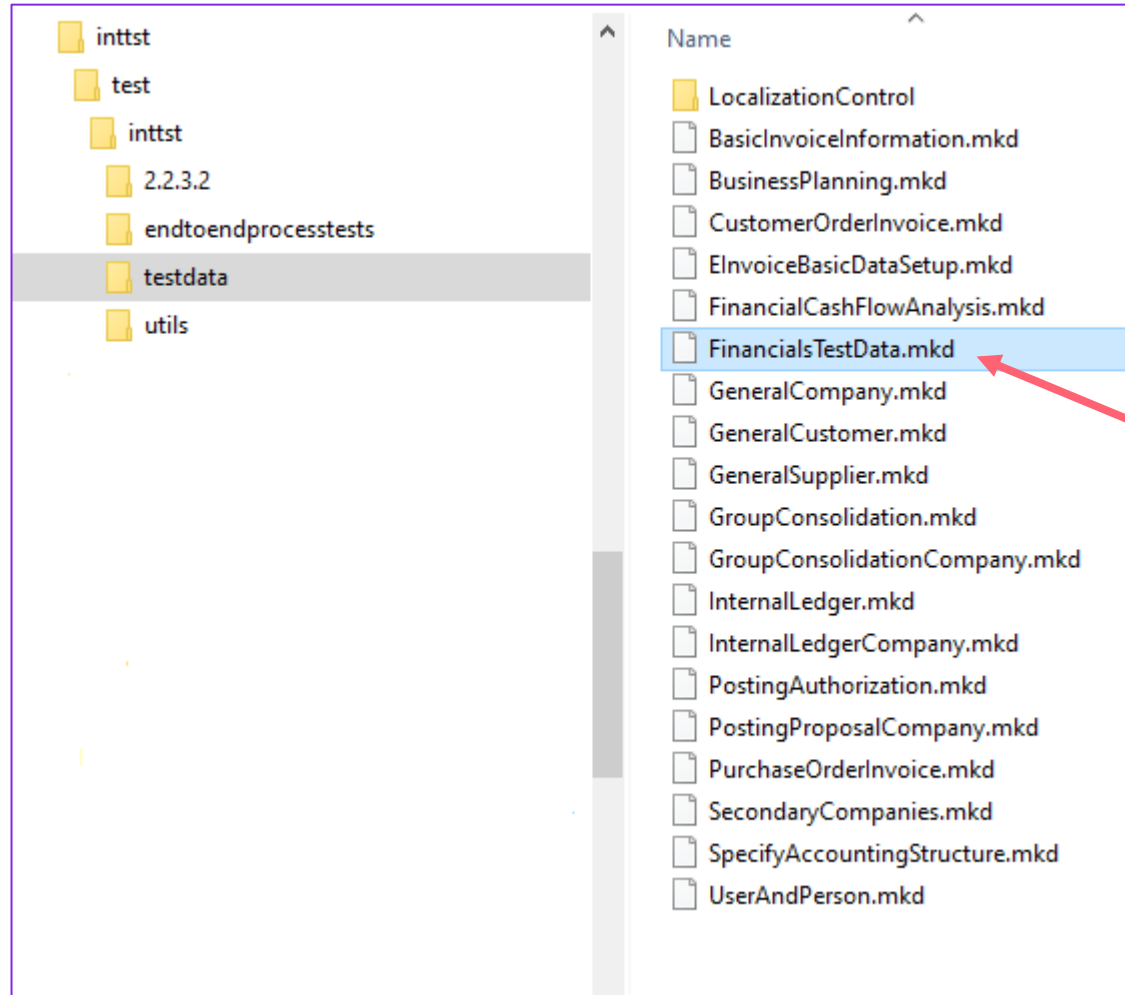
In the code part you express what REST API to call, what data to send and what to Assert. The code part must start with “```cs”, otherwise the code will not be detected. It's possible to have several code blocks in the same document

Code

```
1  ---
2  type: Test-Case
3  owner: [Product-Area]
4  mode: [Standalone/Dependent]
5  ---
6
7  # X.X.X.X [Activity-Diagram-name]
8
9  ## Description
10 What-is-this-Test-Case-about.
11
12 ```cs
13 // Add your code here
14 ```
```

Test Script Structure

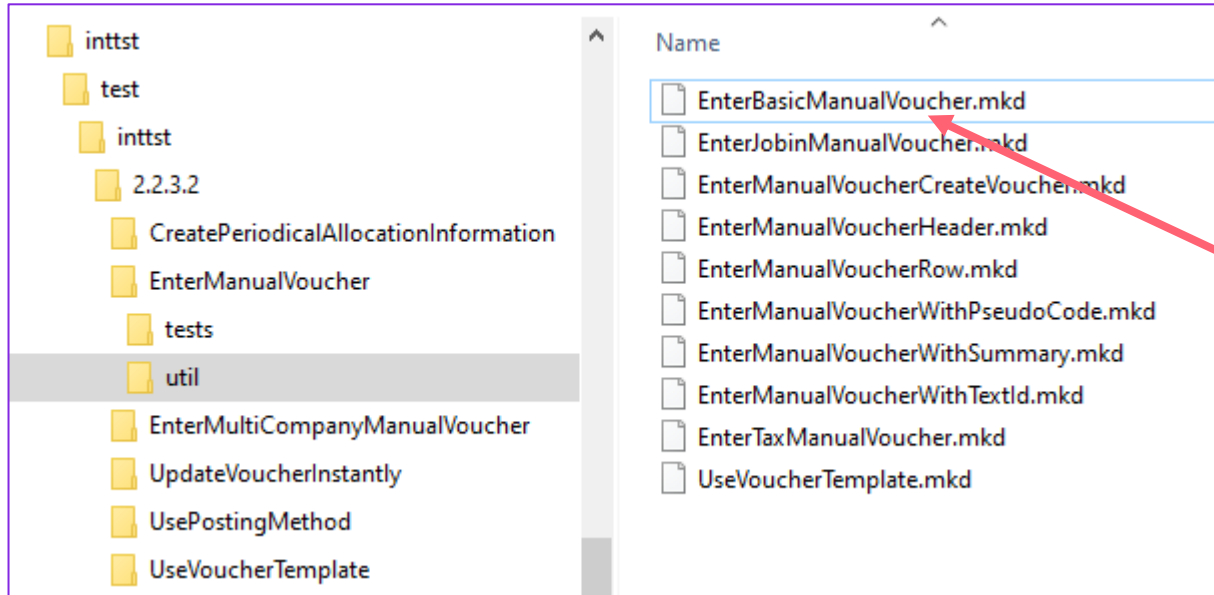
Test Data Template



```
Templates > ProductAreaAllData.md > abc # [Product Area] Test Data
1 ---
2 type: Test Data
3
4 owner: [Product Area]
5
6 mode: [Standalone/Dependent]
7
8 ---
9
10 # [Product Area] Test Data
11
12 ## Test Data Area
```

Test Script Structure

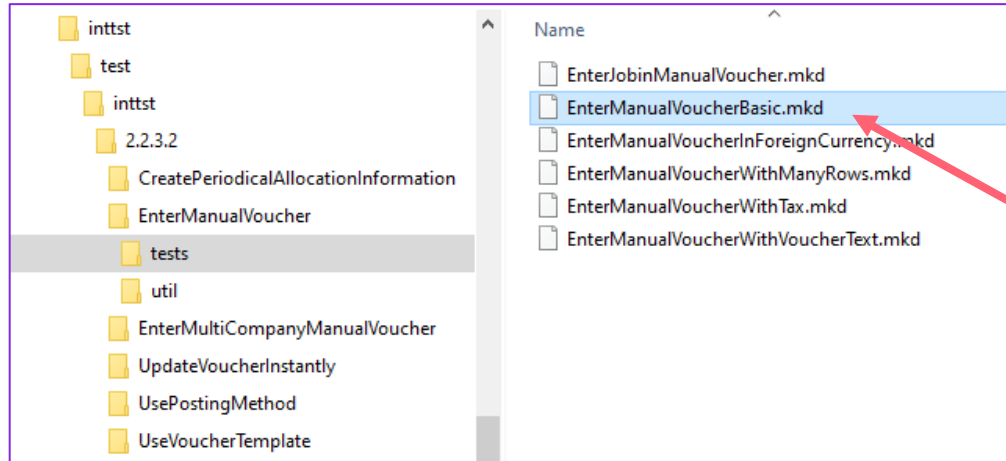
Test Util Template



```
TestUtil.md > ...
1  ---
2  type: Test Util
3  owner: [[Product Area]]
4  ---
5
6  # X.X.X.X [Activity Diagram name] -- [Activity
7  name] -- [Test Util name]
8
9  ## Description
10
11 ## Required input
12
13
14 ```cs
15
16
17 ```
```

Test Script Structure

Test Case Template



```
Templates > TestCase.md > ...
1  ---
2  type: Test Case
3  owner: [Product Area]
4  mode: [Standalone/Dependent]
5  ---
6
7  # X.X.X.X [Activity Diagram name] -- [Activity name] -- [Test Case name]
8
9  ## Description
10
11  ## Arrange
12
13  ## Act
14
15  ## Assert
16
17  ```cs
18  // Add your code here
19  ```
```

Test Script Structure

AAA – Arrange, Act and Assert

Describe what to test and verify

Describe in plain text prerequisites, what you want to test and what you want to verify. See it as a test specification where you need to describe the test case in enough detail so that anyone who is referring the script can have a good understanding.

Arrange

Are there any prerequisites that need to be there before this test can start?

Act

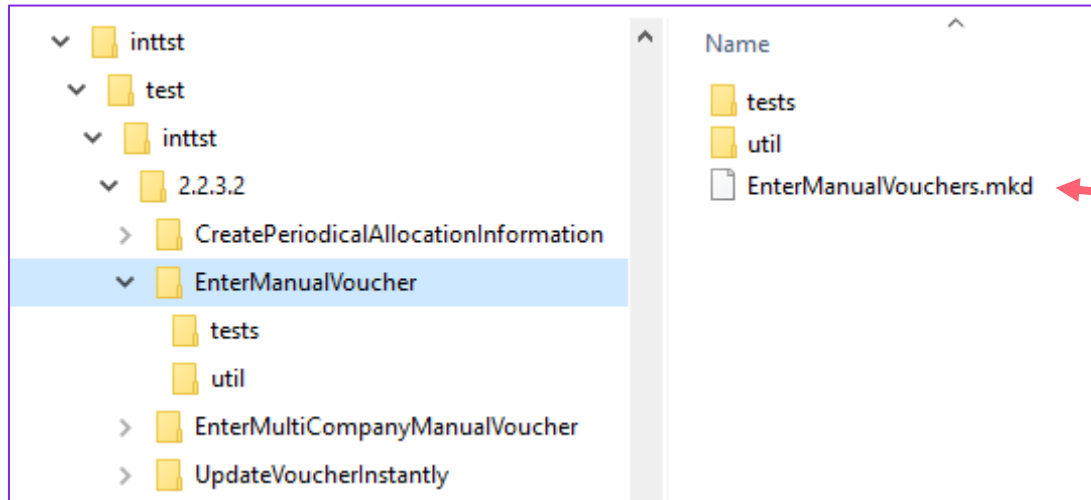
Interact with what you want to verify. This might be creating something, updating something or changing a status.

Assert

Verify the action you did in the Act section. Specify the expected outcome.

Test Script Structure

Test Suite Template



```
ActivityTestSuite.md > abc # X.X.X.X [Activity Diagram name] - [Activity name]
1 ---
2 type: Test Suite
3 owner: [Product Area]
4 mode: [Standalone/Dependent]
5 ---
6
7 # X.X.X.X [Activity Diagram name] -- [Activity name]
8
9 ## Description
10 What is this Test Case about.
11
12
13 ```cs
14
15
16
```

Test-A-Rest Basic Examples



Test-A-Rest Basic Examples

Basic Example

1. Navigate to Manage Geographies > Countries window in Aurena client.
2. Click on New button and enter values for Country Id and Country fields.
3. Open Chrome Dev Tools (Press F12 key) and click on Save button.
4. Observe the network calls from Client to Server in Chrome Dev tools console.
 - Http Post request -
`https://<ApplicationAccessPoint>/main/ifsapplications/projection/v1/EnterpriseHandling.svc/TrnCountries`
 - Request Payload

```
▼ Request Payload    view source
  ▼ {CountryId: "FI", Country: "Finland"}
    Country: "Finland"
    CountryId: "FI"
```

Test-A-Rest Basic Examples

HTTP vs Test-A-Rest Syntax

HTTP Request method	Test-A-Rest Command
POST	Create or Action Create indicates that you want to create a new record. Action is used when you want to invoke a Command
GET	Get
PATCH	Modify
DELETE	Delete

Test-A-Rest Basic Examples

Create a new country record

- Create TrnCountryCreate.md file with following code

```
1  ## Create Country basic data
2
3  ```cs
4
5  Create EnterpriseHandling.svc/TrnCountries
6  {
7      "CountryId": "FI",
8      "Country": "Finland"
9  }
10
11  ```
12
```

Test-A-Rest Basic Examples

Modify an existing country record

- Record to be modified should be uniquely identified before the modification
- This can be done by invoking a Get call and save it in a result variable
- The result variable will hold the version information (captured via Etag value) of the record to ensure the data integrity when performing the modify operation
- Invoke a Modify call using the result variable with changed fields

```
1  ## Modify existing country
2
3  ```cs
4
5  Get EnterpriseHandling.svc/TrnCountries(CountryId='FI') Into result
6
7  Modify EnterpriseHandling.svc/TrnCountries(CountryId='FI') using result
8  {
9      "Country": "Suomi"
10 }
11
12 ```
13
```

Test-A-Rest Basic Examples

Remove an existing country record

- Record to be removed should be uniquely identified before the modification
- This can be done by invoking a Get call and save it in a result variable
- The result variable will hold the version information (captured via Etag value) of the record to ensure the data integrity when performing the remove operation
- Invoke a Delete call using the result variable without any payload

```
1  ## Delete existing country
2
3  ```cs
4
5  Get EnterpriseHandling.svc/TrnCountries(CountryId='FI') Into result
6
7  Delete EnterpriseHandling.svc/TrnCountries(CountryId='FI') using result
8
9  ```
10
```

Test-A-Rest Basic Examples

Assertions

- Assertions can be used to verify whether the business flow work as expected by comparing the result with the desired output
- Assertions are used in Test Cases.

```
1  ## Modify existing country with Assertion
2
3  ```cs
4
5  Get EnterpriseHandling.svc/TrnCountries(CountryId='FI') Into result
6
7  Modify EnterpriseHandling.svc/TrnCountries(CountryId='FI') using result Into modifiedRecord
8  {
9      "Country":"Suomi"
10 }
11
12 Assert modifiedRecord.Country == "Suomi"
13
14 ```
15
```

Test-A-Rest Basic Examples

Variables

- Can be reused within the script providing a better structure to the code
- Eval command is used to assign values to variables
- Use `{${<Variable Name>}}` to get the value from the variable

```
1  ## Create a new country and modify properties via variables and perform assertions
2
3  ```cs
4
5  Eval "FI" Into countryId
6  Eval "Suomi" Into traditionalName
7  Eval "Finland" Into internationalName
8
9  Create EnterpriseHandling.svc/TrnCountries Into newCountryRecord
10 {
11     "CountryId": "${countryId}",
12     "Country": "${internationalName}"
13 }
14
15 Modify EnterpriseHandling.svc/TrnCountries(CountryId='${countryId}') using newCountryRecord Into modifiedRecord
16 {
17     "Country": "${traditionalName}"
18 }
19
20 Assert modifiedRecord.Country == "${traditionalName}"
21
22 ```
```

Test-A-Rest Script

Calling other scripts

caller

```
## This is a Basic Example for Calling Scripts

```cs

Eval "NPYTRANUSD" Into company
Eval "DEMO" Into projectId
Call util/EnterProjectInfo.md With json Into response
{
 "Company": "{$company}",
 "ProjectId": "{$projectId}",
 "Description": "Demo Payment",
 "ProjectGroup": "20",
 "ValidFrom": "2020-02-24",
 "ValidUntil": "2049-12-31",
 "ProjectOrigin": "FinancialProject",
 "CapitalizationPostMethod": "Periodical",
 "BudgetPlanningOnly": false,
 "BudgetPlanningOnlyDb": false,
 "CodeStringCompletionExists": false,
 "AttributeExists": false
}

Assert "{$response.ProjectStatus}" == "Active"

```
```

callee

```
## This is a Basic Example to demo util concept

```cs

Create FinancialProjectHandling.svc/AccountingProjectSet Into projInfo
{$input}

output projInfo

...

 "ProjectOrigin": "{$input.ProjectOrigin}",
 "CapitalizationPostMethod": "{$input.CapitalizationPostMethod}",
 "SpecificCapPrinciples": {$input.SpecificCapPrinciples},
 "BudgetPlanningOnly": {$input.BudgetPlanningOnly},
 "BudgetPlanningOnlyDb": {$input.BudgetPlanningOnlyDb},
 "CodeStringCompletionExists": {$input.CodeStringCompletionExists},
 "AttributeExists": {$input.AttributeExists}
}
output projInfo

```
```

JSON Manipulation



JSON Manipulation

Overview

- A JSON object can be represented in following manner

testVar1

```
{  
  "s1": "Hello",  
  "s2": "World",  
  "n1": 3.14  
}
```

- Reusing the defined JSON object in different REST calls
- Perform different operations using following keywords
 - ApplyJson
 - CopyJson
 - RemoveJson
 - AssertJson

JSON Manipulation

ApplyJson

```
ApplyJson Into defaultValues
{
    "BudgetPlanningOnly":false,
    "BudgetPlanningOnlyDb":false,
    "CodeStringCompletionExists":false,
    "AttributeExists":false
}

ApplyJson Using defaultValues Into record
{
    "Company":"NPYTRANUSD",
    "ProjectId":"DEMO",
    "Description":"Demo Payment",
    "ProjectGroup":"20",
    "ValidFrom":"2020-02-24",
    "ValidUntil":"2049-12-31",
    "ProjectOrigin":"FinancialProject",
    "CapitalizationPostMethod":"Periodical"
}

Create FinancialProjectHandling.svc/AccountingProjectSet
{$record}
```

- Can be used to create a new variable holding JSON data
- Create and append JSON content to a new variable based on an existing JSON variable
- Here, all elements of the JSON object is applied to the other variable.

JSON Manipulation

CopyJson

- Copy specific JSON elements from a JSON object variable to another

```
ApplyJson Into defaultValues
```

```
{  
  "BudgetPlanningOnly":false,  
  "BudgetPlanningOnlyDb":false,  
  "CodeStringCompletionExists":false,  
  "AttributeExists":false  
}
```

```
CopyJson BudgetPlanningOnly, BudgetPlanningOnlyDb, CodeStringCompletionExists, AttributeExists Using defaultValues Into defaultRecord
```

JSON Manipulation

RemoveJson

- Remove specified JSON values from a JSON object variable
- Specified JSON values that do not exist in the variable will be ignored

testVar1

```
{  
  "s1": "Hello",  
  "s2": "World",  
  "n1": 3.14  
}
```

RemoveJson s1,s2 Using testVar1 Into testVar2

JSON Manipulation

AssertJson

- Assert a Json structure towards an expected Json structure, and raise error if the expected Json is not equal to or a subset of the expected Json structure

AssertJson json1 Using json2

Questions?

Thank you!



© COPYRIGHT© 2021 BY INDUSTRIAL AND FINANCIAL SYSTEMS, IFS AB (PUBL). ALL RIGHTS RESERVED. THIS MATERIAL AND ITS CONTENT IS PRODUCED BY THE IFS ACADEMY FOR AUTHORIZED TRAINING PURPOSES ONLY AND REMAINS THE INTELLECTUAL PROPERTY OF IFS. NEITHER THE MATERIAL OR ITS CONTENT MAY BE COPIED, REPRODUCED, OR DISTRIBUTED WITHOUT IFS' EXPRESS WRITTEN PERMISSION.

IFS DOES NOT WARRANT, EITHER EXPRESSLY OR IMPLIED, THE ACCURACY, TIMELINESS, OR APPROPRIATENESS OF THE INFORMATION CONTAINED IN THIS TRAINING MATERIAL AND DISCLAIMS ANY RESPONSIBILITY FOR CONTENT ERRORS, OMISSIONS, OR INFRINGING MATERIAL. IFS ALSO DISCLAIMS ANY RESPONSIBILITY ASSOCIATED WITH RELYING ON THE INFORMATION PROVIDED IN THIS DOCUMENT AND ANY AND ALL LIABILITY FOR ANY MATERIAL CONTAINED ON OTHER CHANNELS THAT MAY BE LINKED TO THE IFS TRAINING MATERIAL.